Seismic CMD

- Course name: Laboratory of Advanced Programming
- Date: 1 April 2026
- Group name: ALFA
  - Team Members:
    Maria Flavia Pontoni 2075265,
    Alessandro Mangiucca 2272857,
    Alessia Angelé 2003000,
    Leonardo Vona 2002043.

Problem Context

Geopolitical Scenario

Sensitive seismic intelligence monitoring requires distributed architecture with neutral-region broker placement

System Objective

Real-time ingestion, frequency-domain analysis, event classification with continuous dashboard monitoring

Core Constraints

Neutral broker cannot process intelligence data. Processing replicas must recover automatically from failures

User Stories Overview

01

System Status & Monitoring

- stream connection status indicator
- replica health monitoring
- UTC system clock
- real-time system logs

02

Event Summary & Alerts

- event counters per category
- nuclear alert notification
- real-time anomaly awareness

03

Data Visualization & Historical Storage

- frequency distribution chart
- historical event storage
- detailed event metadata

04

Dynamic Filtering

- filter by event type
- filter by region
- filter by sensor
- empty-state feedback

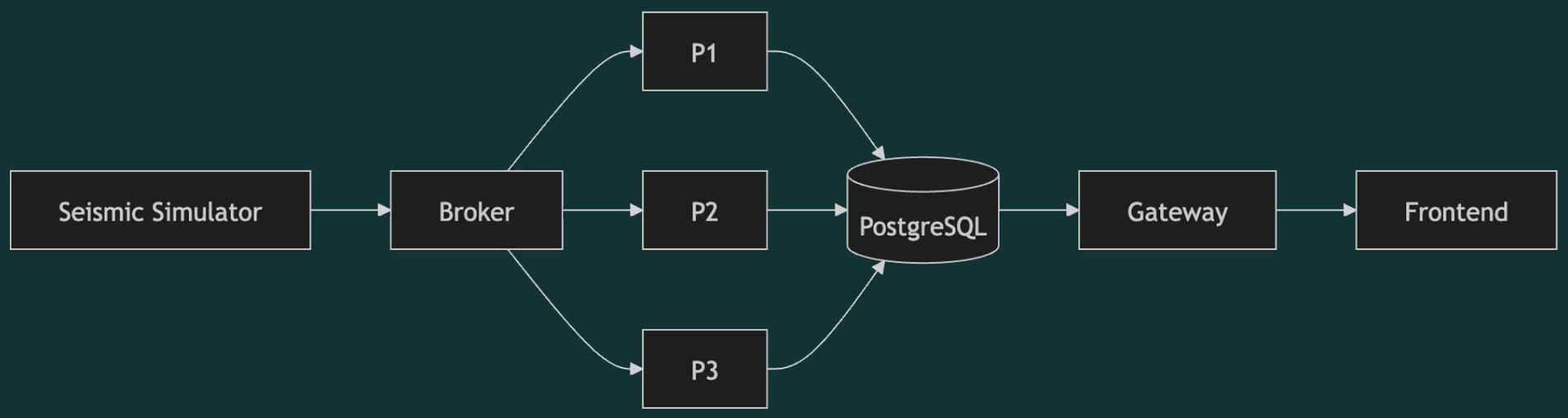
05

Real-Time Event Feed

- live event cards
- event technical details
- filtering-aware updates

Comprehensive list of User Stories: https://github.com/angelealessia/2003000_alfa/blob/main/input.md

Architecture Overview



1. Simulator

The simulator generates real-time seismic signals coming from geographically distributed sensors. Each sensor produces a continuous stream of measurements delivered through WebSocket connections.

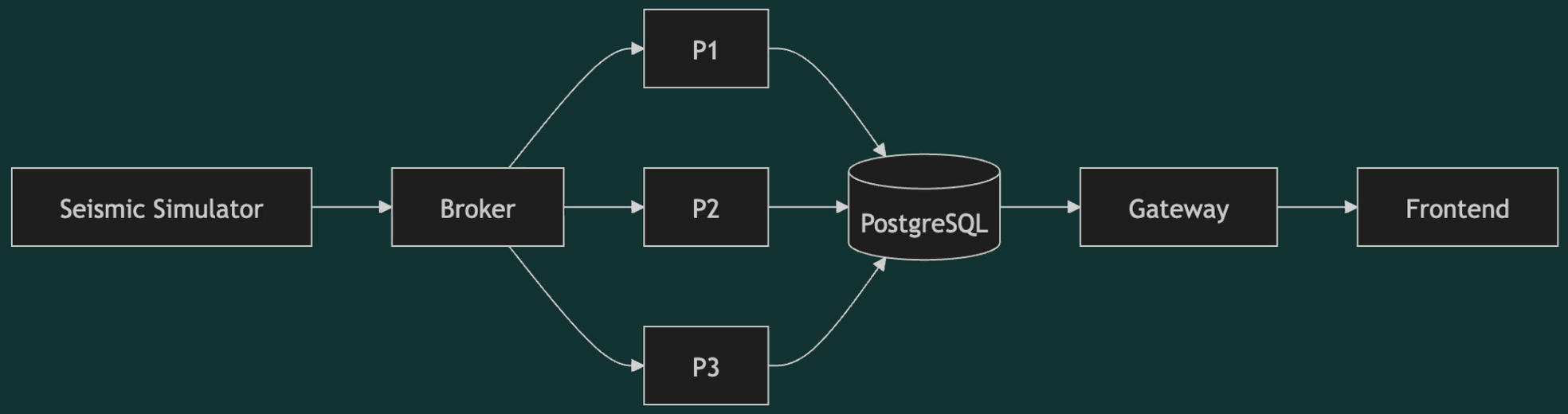
2. Broker

The broker acts as a fan-out component. It receives the incoming measurements from the simulator and redistributes each message to all available processing replicas. The broker performs no data transformation or analysis. Its only responsibility is message routing.

3. Processing Replicas (P1, P2, P3)

Each processing replica maintains an in-memory sliding window of recent sensor measurements. When the window is full, the replica applies Fast Fourier Transform (FFT) to detect the dominant frequency component of the signal. Based on the dominant frequency, the event is classified into: Earthquake, Explosion, Nuclear-like event. Each replica processes the same input stream, ensuring fault tolerance in case one replica fails.

Architecture Overview



4. PostgreSQL Database

All detected events are stored in a shared PostgreSQL database. The database ensures duplicate-safe persistence using a UNIQUE constraint on sensor_id and timestamp. This guarantees idempotent writes when multiple replicas process identical data.

5. Gateway

The gateway provides a single entry point for the frontend. It routes requests to healthy processing replicas using a round-robin strategy. The gateway also exposes a Server-Sent Events (SSE) endpoint to deliver real-time updates.

6. Frontend Dashboard

The frontend displays detected seismic events in real time. It provides visualization tools such as event lists, frequency charts, and replica health status.

Data Flow Pipeline

1

WebSocket Ingestion

Simulator establishes persistent connections

2

Broker Fan-Out

Broadcast distribution to replicated services

3

FFT Analysis

Frequency-domain signal processing

4

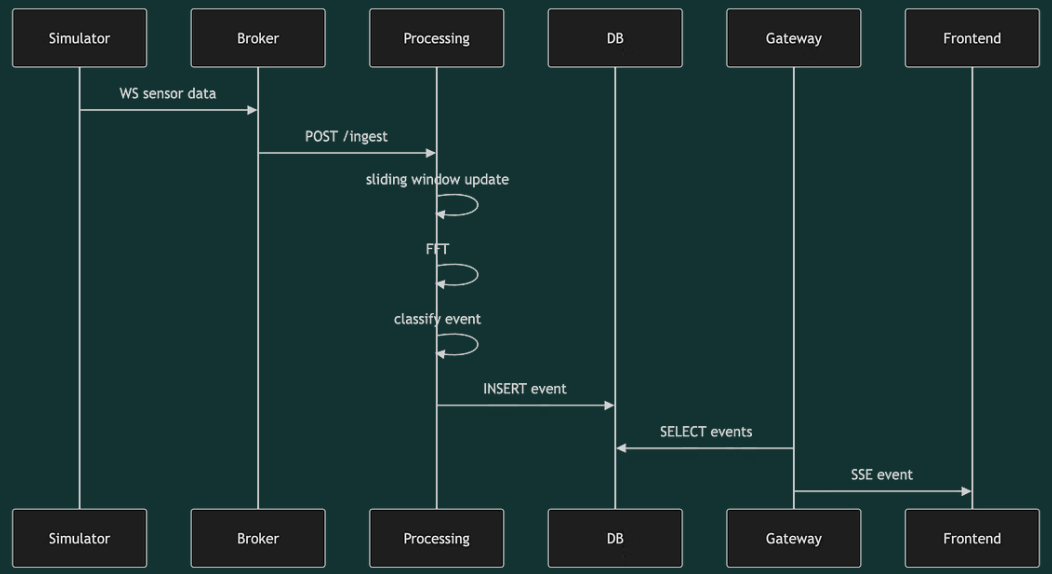
Idempotent Persistence

Idempotent event storage

5

SSE Updates

Real-time dashboard streaming



Backend Design

Broker Responsibilities

- persistent WebSocket connections
- fan-out distribution to processing replicas
- neutral-region deployment constraint
- no intelligence data processing

Broker Fan-Out Implementation

```
async def fan_out_worker():
    async with httpx.AsyncClient(timeout=2.0) as client:
        while True:
            measurement = await measurement_queue.get()
            tasks = [client.post(f"{replica}/ingest", json=measurement) for replica in PROCESSING_REPLICAS]
            results = await asyncio.gather(*tasks, return_exceptions=True)
            for replica_url, result in zip(PROCESSING_REPLICAS, results):
                if isinstance(result, Exception):
                    logger.debug(f"Replica {replica_url} unreachable: {result}")
```

Backend Design

Processing Replica Logic

- sliding window analysis implementation
- FFT dominant frequency extraction
- frequency-based event classification
- idempotent database write operations

FFT Analysis & Event Classification

```
def analyze_window(samples):
    arr = np.array(samples)
    arr -= arr.mean()
    fft_vals = np.abs(np.fft.rfft(arr))
    freqs = np.fft.rfftfreq(n, d=1.0 / SAMPLING_RATE)
    dominant_freq = float(freqs[np.argmax(fft_vals)])
    return dominant_freq, classify_event(dominant_freq)

def classify_event(freq):
    if 0.5 <= freq < 3.0: return "EARTHQUAKE"
    if 3.0 <= freq < 8.0: return "EXPLOSION"
    if freq >= 8.0: return "NUCLEAR"
```

Persistence and Idempotency

In a broadcast-based distributed architecture, all replicas process the same data stream, making idempotent persistence essential; the UNIQUE(sensor_id, detected_at) constraint ensures consistency and prevents duplicate records across concurrent processing nodes.

```
CREATE UNIQUE INDEX
ON events
(sensor_id, detected_at);

INSERT INTO measurements
VALUES (...)
ON CONFLICT DO NOTHING;
```

Fault Tolerance Strategy

Replica Shutdown Detection

SSE control stream triggers replica termination

Automatic Replicas Restart

Docker restart policy recreates failed instances

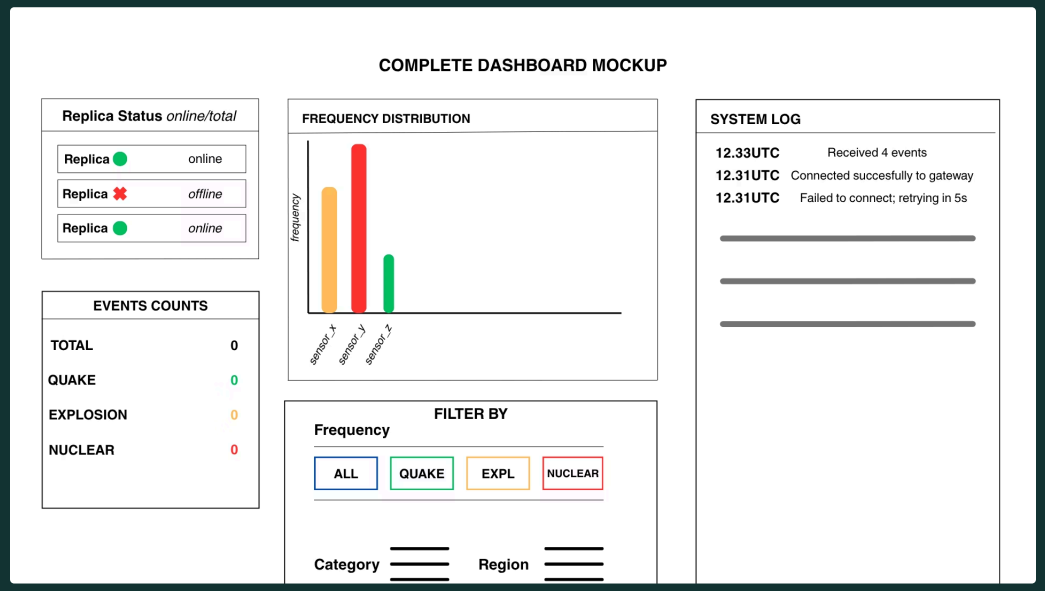
Gateway Health Checks

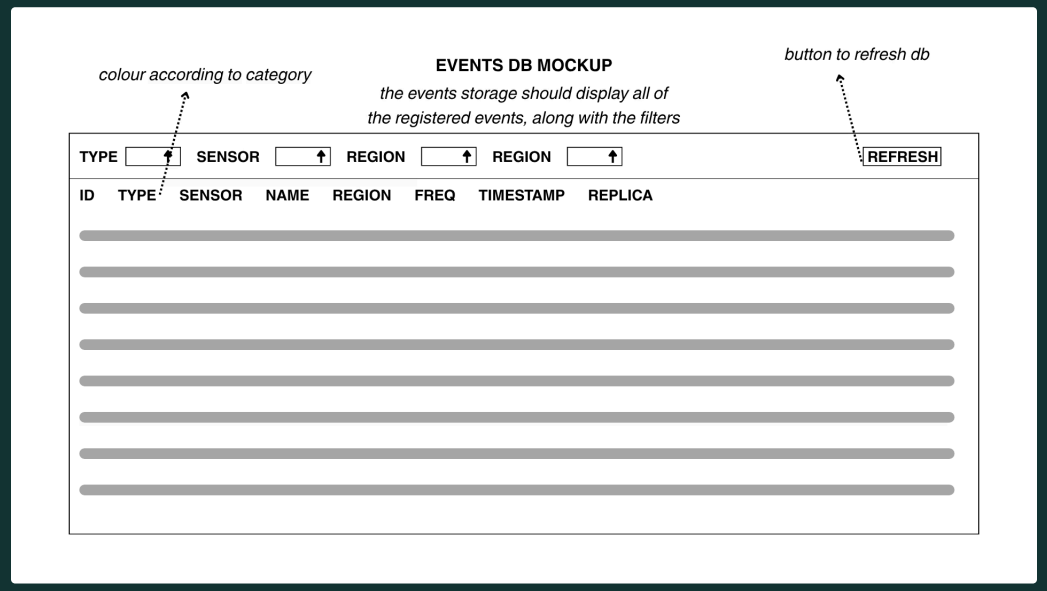
Gateway routes requests only to healthy replicas

```
async def health_checker():
    async with httpx.AsyncClient(timeout=2.0) as client:
        while True:
            for replica in REPLICAS:
                try:
                    resp = await client.get(f"{replica}/health")
                    is_healthy = resp.status_code == 200
                except Exception:
                    is_healthy = False
                replica_health[replica] = is_healthy
            await asyncio.sleep(HEALTH_CHECK_INTERVAL)

def next_replica() -> str | None:
    healthy = get_healthy_replicas()
    return healthy[_rr_index % len(healthy)] if healthy else None
```

LoFi Mockups





LoFi Mockups

FREQUENCY GRAPH MOCKUP

the graph shows the frequency of events along with the sensor that detected them. the colour should depend on the event category

hovering a column should show the exact frequency

EVENTS COUNTER MOCKUP

the table shows the number of total events and for each category

EVENTS COUNTS	
TOTAL	0
QUAKE	0
EXPLOSION	0
NUCLEAR	0

the colour depends on the event category like in the graph

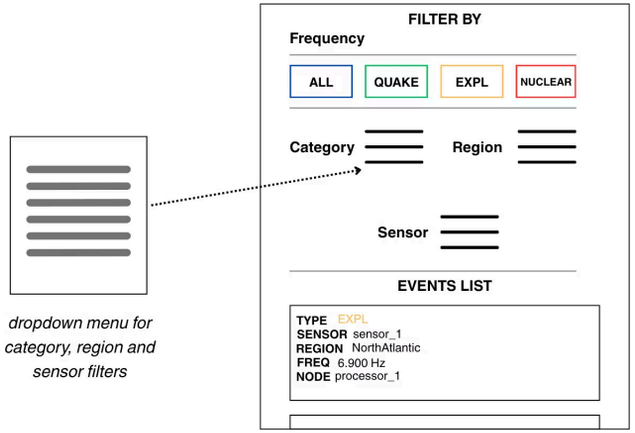
REPLICA STATUS SECTION MOCKUP

each replica should be displayed with its name and status

Replica Status *online/total*	
Replica	online
Replica	*offline*
Replica	*online*

FILTERS MOCKUP

the user should be able to filter events by frequency, category, region and sensor



dropdown menu for category, region and sensor filters

SYSTEM LOG MOCKUP

the log should keep track of received events and connection status alerts

SYSTEM LOG

12.33UTC	Received 4 events
12.31UTC	Connected succesfully to gateway
12.31UTC	Failed to connect; retrying in 5s

Frontend Dashboard

Frequency Distribution

Live FFT analysis visualization with dominant frequency tracking

Replica Status Monitoring

Real-time monitoring of processing service availability

Event Filtering Interface

Interactive filtering by type, sensor, region

Conclusion

Distributed Architecture

Microservices system with neutral-region broker

Fault Tolerance

Automatic replica recovery via Docker restart

Idempotent Persistence

Duplicate-safe storage using UNIQUE constraints

Real-Time Monitoring

Real-time dashboard via SSE event streaming